Actually, it's not strictly necessary to compile the code first with `scalac`. A simple program like the one we've written here can be run using just the Scala interpreter by passing it to the `scala` command-line tool directly:

```
> scala MyModule.scala
The absolute value of -42 is 42.
```

This can be handy when using Scala for scripting. The interpreter will look for any object within the file `MyModule.scala` that has a `main` method with the appropriate signature, and will then call it.

Lastly, an alternative way is to start the Scala interpreter's interactive mode, the REPL (which stands for read-evaluate-print loop). It's a great idea to have a REPL window open so you can try things out while you're programming in Scala.

We can load our source file into the REPL and try things out (your actual console output may differ slightly):

```
> scala
Welcome to Scala.
Type in expressions to have them evaluated.
Type :help for more information.

scala> :load MyModule.scala
Loading MyModule.scala...
defined module MyModule

scala> MyModule.abs(-42)
res0: Int = 42
```

We can type Scala expressions at the prompt.

`:load` is a command to the REPL to interpret a Scala source file. (Note that unfortunately this won't work for Scala files with package declarations.)

The REPL evaluates our Scala expression and prints the answer. It also gives the answer a name, `res0`, that we can refer to later, and shows its type, which in this case is `Int`.

It's also possible to copy and paste individual lines of code into the REPL. It even has a paste mode (accessed with the `:paste` command) designed to paste code that spans multiple lines. It's a good idea to get familiar with the REPL and its features because it's a tool that you'll use a lot as a Scala programmer.

2.3 Modules, objects, and namespaces

In this section, we'll discuss some additional aspects of Scala's syntax related to modules, objects, and namespaces. In the preceding REPL session, note that in order to refer to our `abs` method, we had to say `MyModule.abs` because `abs` was defined in the `MyModule` object. We say that `MyModule` is its *namespace*. Aside from some technicalities, every value in Scala is what's called an *object*,[1] and each object may have zero or more *members*. An object whose primary purpose is giving its members a namespace is sometimes called a *module*. A member can be a method declared with the `def` keyword, or it can be another object declared with `val` or `object`. Objects can also have other kinds of members that we'll ignore for now.

We access the members of objects with the typical object-oriented dot notation, which is a namespace (the name that refers to the object) followed by a dot (the period

[1] Unlike Java, values of primitive types like `Int` are also considered objects for the purposes of this discussion.